

SemiticGPT: A Low-Cost Recipe for Multilingual Foundation Models in an Under-Resourced Semitic-Centered Language Cluster

Ronnen Slasky¹

¹Independent Researcher

ronnen.slasky@gmail.com

GitHub: [semitic-gpt](#) · GitHub: [fatherRonnen](#)

April 2026

ABSTRACT

We present a practical blueprint for building a multilingual foundation model from scratch for an under-resourced language family, demonstrated through SemiticGPT: a 3-billion parameter decoder-only model covering Hebrew, Arabic, Farsi, and English trained on approximately 20 billion tokens using cloud spot instances. Our central finding is that multilingual pretraining with a linguistically-motivated language cluster produces **meaningful cross-lingual semantic transfer** between related languages, but does not yield robust translation or abstract reasoning. Specifically, fine-tuning on Hebrew sentiment data alone improves Arabic sentiment accuracy from 5.5% to 49% (with zero Arabic task data), while Farsi—which shares Arabic script but belongs to a different language family—shows no comparable transfer (0.5%→1.5%). This suggests that in our setup, **linguistic family relatedness matters more than script similarity** for semantic transfer. We also report instructive negative results: English-mediated parallel data does not enable direct translation between non-English pairs, and all-language fine-tuning underperforms focused same-family fine-tuning. We provide a step-by-step practitioner’s guide and release the model, tokenizer, and training code.¹

Keywords: multilingual language models, cross-lingual transfer, Hebrew, Arabic, Farsi, Semitic languages, tokenizer design, low-resource NLP, sentiment transfer, language family clustering, BPE tokenization, practitioner recipe

1. Introduction

Large language models have achieved remarkable capabilities across a wide range of natural language tasks, yet their development has been overwhelmingly concentrated on English and other high-resource languages [Joshi et al., 2020]. Languages of the Semitic family—particularly Hebrew and Arabic—remain underserved despite their combined speaker population exceeding 400 million. While recent efforts such as Jais [Sengupta et al., 2023] have addressed Arabic, no existing model provides joint coverage of Hebrew, Arabic, and Farsi within a single architecture trained from scratch.

This gap matters not only for these specific languages but as a general problem: dozens of language families worldwide lack dedicated foundation models, and researchers in those communities face similar questions. *How should one design a tokenizer for multiple scripts? How much data per language is needed? What transfers cross-lingually and what does not? Where should limited compute be spent?*

This paper provides empirical evidence toward answering these questions in the context of a Semitic-centered lan-

guage cluster. Specifically, we investigate:

1. Does a balanced custom tokenizer provide materially better encoding efficiency than repurposing an English-centric tokenizer for Hebrew/Arabic/Farsi?
2. Does over-representing an anchor language (Hebrew) in pretraining induce useful cross-lingual transfer to related languages?
3. Does linguistic family relatedness predict semantic transfer better than script similarity in our setup?
4. What capabilities emerge at 3B scale and what do not?

Building on our prior work on autonomous training recipe discovery for Hebrew [Slasky, 2026], we extend from a monolingual setting to a four-language Semitic-family model. The training recipe was validated through proxy experiments at small scale before committing to full pretraining, following the autoresearch methodology [Karpathy, 2026].

Hebrew and Arabic share the Semitic triconsonantal root system, similar derivational morphology (binyan/wazn patterns), and flexible word order. Farsi, while using Arabic script and containing extensive Arabic loanwords, belongs to the Indo-European family—providing a natural control condition for studying linguistic relatedness versus surface-level script similarity.

¹Code and artifacts: <https://github.com/fatherRonnen/semitic-gpt>

Our primary contributions:

1. A **low-cost, reproducible recipe** for training a 3B multilingual decoder-only model from scratch for an under-resourced Semitic-centered language cluster.
2. Evidence that multilingual pretraining with a **custom balanced tokenizer** yields strong cross-lingual retrieval and semantic transfer for linguistically related languages.
3. A finding that **direct parallel data is necessary** for useful direct translation between non-English pairs.
4. A **practitioner’s guide** (Section 6) with step-by-step recommendations including negative results.

2. Related Work

Arabic Language Models. Jais [Sengupta et al., 2023] is a 13B parameter bilingual Arabic-English model trained on 395 billion tokens. AraGPT2 [Antoun et al., 2021] provided GPT-2 scale Arabic models. ArabianGPT [Koubâa and Ammar, 2024] focuses on native Arabic. None include Hebrew or Farsi.

Hebrew-Arabic Models. HeArBERT [Rivas et al., 2024] explores bilingual Arabic-Hebrew modeling through transliteration-based script unification but is limited to encoder-only (BERT) architecture. To our knowledge, no prior work has trained a generative decoder-only model jointly on Hebrew, Arabic, and Farsi from scratch.

Small Multilingual Models. SmolLM3 [SmolLM3, 2025] provides a detailed 3B parameter training recipe but targets European languages and does not address non-Latin script languages or Semitic-family challenges.

Multilingual Adaptation. Marco-LLM [Marco-LLM, 2024] extends Qwen2 to low-resource languages through continual pre-training, inheriting the base model’s potentially suboptimal tokenizer.

Our work differs by (1) training from scratch with a custom tokenizer, (2) combining Hebrew, Arabic, and Farsi, (3) operating at academic budget, and (4) providing comprehensive cross-lingual transfer evaluation.

Training Recipe Discovery. Our recipe was derived via the autoresearch framework [Karpathy, 2026]. In our prior work [Slasky, 2026], we conducted 200 proxy experiments for Hebrew, discovering that WSD linear-decay outperforms cosine, Muon outperforms AdamW at 1B but fails at larger multilingual scale, and SWA benefits Muon but degrades AdamW. The multilingual recipe was validated through 32 additional proxy experiments at 110M scale.²

3. Methodology

3.1 Tokenizer Design

We train a 32,768-token SentencePiece BPE tokenizer on a balanced sample (~25% per language). The resulting fertility rates: ~1.4 for Hebrew, 1.5 for Arabic, 1.6 for Farsi, 1.2 for English—significantly better than mBERT’s WordPiece (2.5+ for Hebrew) or LLaMA’s BPE (3+ for Arabic). Special tokens: `<|user|>`, `<|assistant|>`, `<s>`, `</s>`, `<pad>`.³

3.2 Data Collection and Curation

We collect approximately 20 billion tokens with a Hebrew-anchored distribution:

- **Hebrew (40%):** Wikipedia, OSCAR, news, government docs
- **Arabic (25%):** Wikipedia, OSCAR, news, UN corpus
- **English (20%):** Wikipedia, OpenWebText, books
- **Farsi (15%):** Wikipedia, OSCAR, news corpora

Hebrew is overrepresented to ensure strong anchor-language representations that transfer to related languages. Preprocessing: MinHash deduplication, perplexity filtering, language ID verification.⁴

3.3 Architecture

Parameter	Value
Total parameters	3.04B
Layers	36
Hidden dim	2,560
Attention heads	20 (head dim 128)
Vocab size	32,768
Seq length	2,048
Position enc.	RoPE
Activation	SwiGLU
Normalization	RMSNorm

Table 1: *SemiticGPT architecture.*

3.4 Pretraining

Training on AWS spot instances (L40S 48GB, H100 80GB) with FSDP. Configuration:

- AdamW ($\beta_1=0.9$, $\beta_2=0.95$, $\epsilon=1e-8$)
- LR: $3e-4$ peak, cosine decay to 3% of peak
- Warmup: 2,000 steps; Batch: 512K tokens
- Weight decay: 0.1; Gradient clip: 1.0; fp16

Schedule note: Our 3B model uses cosine decay. Post-hoc proxy experiments found WSD linear-decay superior; we recommend WSD for future work (Section 6).

Cost: ~400 GPU-hours on H100-equivalent hardware at spot pricing (60–70% discount).⁵

³Tokenizer config and fertility report: `tokenizer/` in the repository.

⁴Collection scripts and manifests: `data/` in the repository.

⁵Training scripts and logs: `pretrain/` in the repository.

²Proxy configs, results, rationale: `proxy/` in the repository.

3.5 Supervised Fine-Tuning

We investigate SFT scaling (E-1K, E-3K, E-5K steps) and data composition: D (all 4 languages), F-no-arfa (HE+EN only), F-only-arfa (AR+FA only). Configuration files for all six variants are in `sft/` in the repository.

4. Evaluation

We evaluate across six dimensions. Table 2 summarizes configurations; Table 3 lists evaluation datasets.

Config	Data / Change	Hypothesis
D-baseline	5K SFT, all 4 langs	Reference
<i>Translation</i>		
G-trans	EN-mediated parallel	EN pivot enables $X \leftrightarrow Y$?
G2-direct	Direct parallel only	Direct data necessary?
<i>Sentiment</i>		
H1	Hebrew sent. only	HE→AR transfer?
H2	All 4 langs sent.	Multilingual better?
H3	AR+FA sent. only	Focused > diluted?
<i>SFT Ablations</i>		
E-1K/3K/5K	Vary SFT steps	Optimal duration
F-no-arfa	HE+EN only	Remove AR/FA effect

Table 2: Experiment legend.

Eval	Dataset	N	C	Source
BPB	Held-out splits	10K	—	Internal
Belebele	Belebele	900	4	Bandarkar et al. [2023]
Translation	OPUS parallel	50	—	OPUS
Retrieval	Parallel sent.	10	—	Tatoeba
Sentiment	Per-lang datasets	200	2	HuggingFace
News	AG News	200	4	AG News

Table 3: Evaluation datasets. N =samples/lang, C =classes.

Methodology. Classification uses generative evaluation (model generates label text, matched via case-insensitive substring). Low baselines (<chance) likely reflect parsing issues; relative improvements remain informative.⁶

4.1 Language Modeling (BPB)

Model	HE	AR	EN	FA
Base	0.879	0.731	0.972	0.663
D-SFT (5K)	0.876	0.726	0.964	0.657
E-1K	0.878	0.729	0.967	0.663
E-3K	0.877	0.728	0.965	0.660
F-no-arfa	0.879	0.733	0.968	0.669
F-only-arfa	0.880	0.727	0.968	0.658

Table 4: BPB across SFT configurations. Lower is better.

SFT consistently improves BPB with diminishing returns after 3K steps. Removing a language from SFT degrades that language.

⁶All scripts, prompts, results: `eval/` in the repository.

4.2 Reading Comprehension (Belebele)

	HE	AR	EN	FA
Random	25.0	25.0	25.0	25.0
Base	28.3	27.3	31.1	29.2
D-SFT	28.1	27.9	31.8	28.6

Table 5: Belebele accuracy (%). Near chance at 3B scale.

4.3 Translation

Key findings: (1) Direct parallel data essential—G2-direct achieves 18.7% for AR→FA. (2) EN-mediated helps $EN \leftrightarrow X$ only. (3) G2-direct collapses on $EN \leftrightarrow X$ (0.4% vs 10.5%). (4) $HE \leftrightarrow AR$ hardest even with direct data (3–5%).

4.4 Cross-lingual Embeddings

Cross-lingual retrieval (10-way, chance=10%):

Direction	Base	D-SFT
EN→HE	90%	90%
HE→EN	90%	80%
AR→EN	70%	50%
AR→HE	70%	70%
EN→AR	50%	60%
FA→HE	50%	30%
HE→AR	40%	60%
HE→FA	40%	30%
EN→FA	20%	20%
AR→FA	10%	10%
FA→EN	10%	10%
FA→AR	10%	10%
Average	45.8%	43.3%

Table 7: Cross-lingual retrieval (10-way, chance=10%). Strong $EN \leftrightarrow HE$ alignment emerges without explicit alignment objective.

$EN \leftrightarrow HE$ achieves 90% ($9 \times$ chance). Farsi shows weakest alignment, consistent with typological distance.

4.5 Cross-lingual Sentiment Transfer

Our most striking result:

Config	HE	AR	FA	EN
No training	19.0	5.5	0.5	11.5
Hebrew only (H1)	18.5	49.0	1.5	33.5
All 4 langs (H2)	18.0	43.0	1.5	18.5
AR+FA only (H3)	23.0	67.0	5.5	21.0

Table 8: Sentiment accuracy (%). Hebrew-only training improves Arabic from 5.5% to 49% ($9 \times$) with zero Arabic sentiment data.

Key findings: (1) **Massive HE→AR transfer:** 5.5%→49% with zero Arabic task data. (2) **Reverse transfer:** AR+FA training improves Hebrew 19%→23%. (3)

	AR→FA	FA→AR	HE→EN	EN→HE	AR→EN	EN→AR	HE→AR	AR→HE	EN→FA	FA→EN
D-baseline	6.2	7.6	1.4	0.9	0.9	0.6	1.4	0.0	0.1	0.3
G-trans	5.4	4.2	10.5	9.1	7.3	5.1	0.0	0.0	3.8	6.0
G-mixed	5.8	6.0	5.7	5.3	4.6	3.3	0.0	0.0	2.5	6.3
G2-trans	16.7	13.7	7.1	5.7	6.0	4.3	1.8	5.9	3.2	5.2
G2-direct	18.7	15.3	0.4	2.2	0.4	3.7	4.8	3.4	2.7	0.4

Table 6: Translation quality (chrF%). G-trans=EN-mediated; G2-direct=direct parallel data. Bold=best per-direction.

Focused > diluted: H3 achieves 67% vs H2’s 43%. (4)
Farsi isolation: 0.5%→5.5% at best despite sharing Arabic script, suggesting linguistic family > script for transfer (though Farsi’s smaller pretraining share may contribute).

4.6 News Topic Classification

4-class (chance=25%): D-baseline 4.0% → after fine-tuning **79.0%**. Strong domain adaptation from limited fine-tuning (500 steps).

5. Analysis and Discussion

5.1 Cost Efficiency

Jais (13B): millions in compute, 395B tokens. SmolLM3 (3B): 11T tokens on dedicated clusters. **SemiticGPT (3B): ~20B tokens, spot instances only.** Linguistically-motivated language grouping amplifies value per GPU-hour.

5.2 What Works vs. What Doesn’t

Works: Emergent cross-lingual embeddings; semantic transfer between related languages; domain fine-tuning with limited data; BPB improvement from SFT.

Does not work: Translation without explicit parallel data; English-mediated translation for $X \leftrightarrow Y$; Farsi transfer despite shared script; reasoning at 3B scale.

5.3 Lessons Learned

1. **Tokenizer balance:** Equal representation prevents over-fragmentation.
2. **Anchor strategy:** Over-representing Hebrew (40%) creates transferable representations.
3. **SFT composition:** Removing languages immediately degrades them.
4. **Direct parallel data non-negotiable** for translation.
5. **fp16:** No observed degradation (no controlled fp32 comparison).

6. Practitioner’s Guide

Step 1: Language Cluster. Prioritize linguistic family over script. Include anchor + control. Recommended: 4–6 languages.

Step 2: Balanced Tokenizer. SentencePiece BPE, equally-weighted, 32K vocab. Target fertility ≤ 1.6 .

Step 3: Pretraining Data. Over-represent anchor (30–40%). MinHash dedup + perplexity filter. Include direct parallel data only for desired translation pairs.

Step 4: Proxy Validation. 20–50 experiments at 100–200M scale. Validate schedule (WSD > cosine), optimizer (AdamW robust; Muon can diverge at scale). Cost: \$10–50.

Step 5: Pretrain. Standard transformer (RoPE, SwiGLU, RMSNorm). Spot instances + checkpointing. Monitor per-language BPB.

Step 6: SFT. All languages, native > translated, 3–5K steps.

Step 7: Evaluate. BPB, cross-lingual retrieval, task transfer, translation, downstream benchmarks.

Decision	Recommendation
Languages	Linguistic family > script
Tokenizer	Custom BPE, 32K, balanced
Data dist.	Anchor at 30–40%
Quality	MinHash + perplexity filter
Validation	20–50 proxy experiments
Optimizer	AdamW ($\beta_2=0.95$)
Schedule	WSD linear-decay
Precision	fp16/bf16
SFT	All langs, native, 3–5K steps
Translation	Direct parallel only
Compute	Spot instances + checkpoint

Table 9: Recipe summary.

7. Conclusion

We have presented SemiticGPT, a 3B parameter multilingual model trained from scratch on cloud spot instances. Our evaluation reveals massive cross-lingual transfer between Semitic languages for semantic tasks (the headline result), alongside instructive negative results about translation and scale limitations.

We offer a reproducible recipe (Section 6) applicable to other language family clusters (Turkic, Bantu, Dravidian). We release model weights, tokenizer, and code.⁷

Future work. (1) Scale to 7B with improved Arabic SFT data; (2) add Semitic languages (Amharic, Tigrinya);

⁷Repository: <https://github.com/fatherRonnen/semitic-gpt>, tag v0.1-paper-draft.

(3) adapter-based language extension; (4) validate transfer at larger scales.

8. Reproducibility

All code and results at <https://github.com/fatherRonnen/semitic-gpt>:

Section	Path	Contents
§3.1	tokenizer/	Config, fertility
§3.2	data/	Scripts, manifests
§2	proxy/	32 configs, results
§3.4	pretrain/	Scripts, logs
§3.5	sft/	6 configs, logs
§4	eval/	Scripts, results
Tables	paper/tables/	JSON data

Table 10: Paper section → repository mapping.

9. Limitations

- **Single-run:** No confidence intervals. Differences <5pp may not be significant.
- **Eval harness:** Baselines below chance reflect parsing issues, not anti-correlation.
- **Small eval sets:** 200 samples/lang detects large but not small effects.
- **Scale:** 3B is below reasoning threshold. Findings may not hold at larger scales.
- **Confounds:** Cannot isolate linguistic family from data proportion (Farsi 15% vs Hebrew 40%).
- **No external baselines:** No mBERT/XLM-R/NLLB comparison.
- **Translation:** Max 18.7% chrF—hard to distinguish from noise.
- **Generalizability:** One language cluster only.

References

- W. Antoun, F. Baly, and H. Hajj. AraGPT2: Pre-trained transformer for Arabic language generation. In *Proc. WANLP*, 2021.
- L. Bandarkar et al. The Belebele benchmark: a parallel reading comprehension dataset in 122 language variants. *arXiv:2308.16884*, 2023.
- P. Joshi, S. Santy, A. Buber, et al. The state and fate of linguistic diversity and inclusion in the NLP world. In *Proc. ACL*, 2020.
- A. Koubâa and A. Ammar. ArabianGPT: Native Arabic GPT-based large language model. *arXiv:2402.15313*, 2024.
- Marco-LLM Team. Marco-LLM: Bridging languages via massive multilingual training for cross-lingual enhancement. *arXiv:2412.04003*, 2024.
- M. Popović. chrF: character n-gram F-score for automatic MT evaluation. In *Proc. WMT*, 2015.
- A. Rivas et al. HeArBERT: A bilingual model for Arabic-Hebrew translation using transliteration. *arXiv:2402.16065*, 2024.
- N. Sengupta et al. Jais and Jais-chat: Arabic-centric foundation and instruction-tuned open generative large language models. *arXiv:2308.16149*, 2023.
- L.B. Allal et al. SmolLM3: The complete blueprint of a state-of-the-art 3B parameter LLM. HuggingFace Technical Report, 2025.
- A. Karpathy. autoresearch: Autonomous AI-driven ML experimentation. GitHub repository, 2026.
- R. Slasky. Autonomous AI-driven Hebrew language model optimization: From random search to optimizer-architecture co-discovery. Independent research report, 2026.